

Systèmes informatiques pour l'IoT
Architecture de services

Compte rendu de projet

Projet Vacow pour la gestion automatisée d'une salle de traite

Jean-Philippe Loubejac Combalbert
Célian Hilal Hamdan

Année scolaire 2025 - 2026
Remis le 21 janvier 2026



INNOVATIVE SMART SYSTEMS



I. Introduction

A. Contexte

Le projet Vacow se place dans le cadre de l'élevage des vaches laitières, et plus précisément dans l'automatisation de la traite. Cette étape de la production se déroule généralement dans une « salle de traite » (voir Figure 1), où le flux de circulation est organisé de la manière suivante : les vaches entrent par un accès dédié, sont installées pour la traite, puis ressortent par une porte opposée.

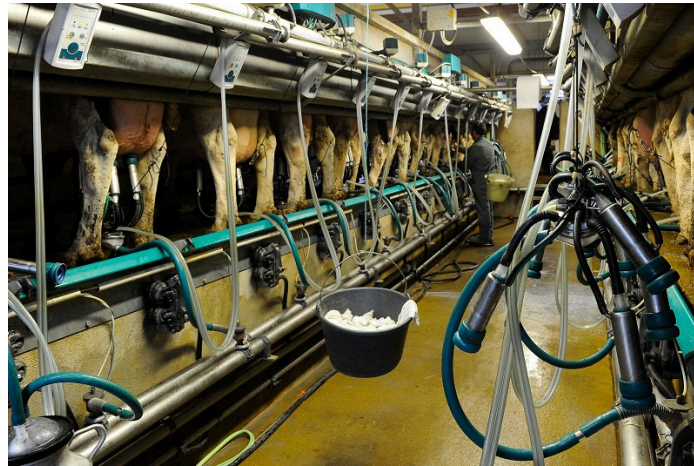


Figure 1 : Salle de traite.

Actuellement, l'opération repose sur l'utilisation d'une griffe de traite (voir Figure 2) pour recueillir le lait. Ce processus est majoritairement manuel : l'éleveur doit lui-même positionner la griffe, surveiller le débit de lait pour s'assurer du bon déroulement de l'opération, puis retirer l'équipement une fois la vache traite. Cette surveillance constante représente une charge de travail importante et répétitive pour l'éleveur.



Figure 2 : Griffe de traite.

B. Problématique

L'objectif de ce projet est d'automatiser une partie du cycle de traite des vaches afin de gagner en efficacité et de libérer du temps pour l'éleveur. Pour répondre à ce besoin, le système comporte plusieurs dispositifs.

Un capteur de débit est disponible par poste de traite. Ces capteurs mesurent en temps réel la quantité de lait extraite. Si le débit descend sous un certain seuil, le système détecte la fin de la traite et active des actionneurs pour retirer automatiquement la griffe.

Des capteurs de présence sont placés aux entrées et sorties, ils permettent de suivre le mouvement des vaches dans la salle.

Une interface de contrôle (Écran LCD) est également présente : elle donne une visibilité complète sur l'état de la salle (température ambiante) et sur l'identité des animaux présents (nom, identifiant, statut de traite).

Finalement, un haut-parleur sert à la fois d'interface pour l'éleveur (alertes, informations) et pour donner des ordres aux vaches afin qu'elles sortent de la salle de traite.

La Figure 3 schématise la salle de traite connectée :

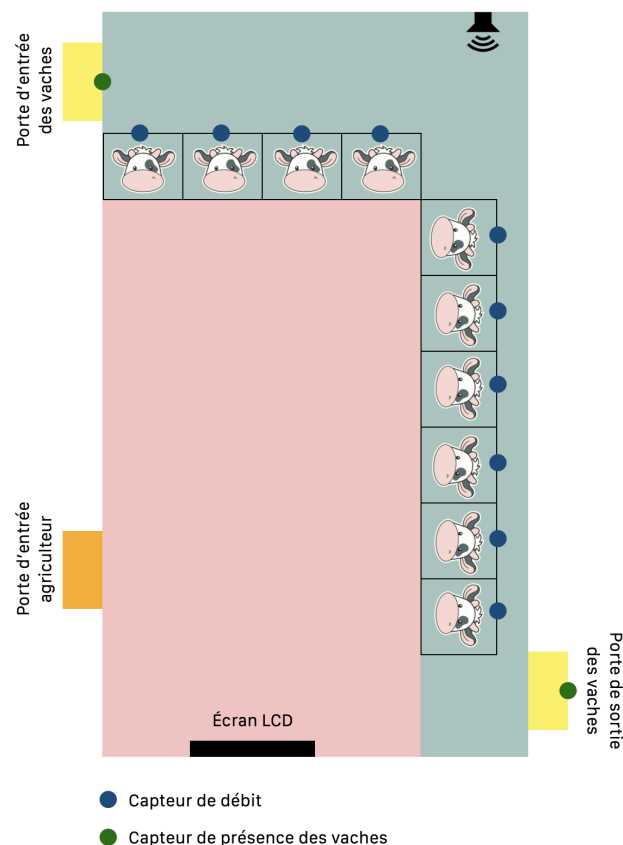


Figure 3 : Schéma d'une salle de traite avec tous les capteurs et actionneurs.

II. Architecture de la base de données

L'architecture de la base de données est présentée à la Figure 4. Elle comporte les tables suivantes :

Vache : Cette table contient toutes les vaches avec leurs identifiants qui représente le numéro national de la vache, leurs noms, leurs dates de naissance, leurs races et un booléen qui indique si la vache est à traire ou pas. Cet attribut est utile pour l'éleveur pour qu'il ne traite pas une vache une seconde fois, mais surtout pour ne pas traire une vache qui ne doit pas l'être (il peut y avoir plusieurs raisons, notamment la prise d'antibiotiques, un vêlage ou des maladies comme une mammite).

Capteurs : C'est une table qui représente tous les capteurs présents. Elle est composée d'un ID, d'un type de capteur et d'une unité de mesure.

Mesures : Cette table enregistre toutes les mesures prises par les capteurs. Elle est identifiée par un ID, par un capteur et une vache, mais aussi par la valeur mesurée, et elle est horodatée.

Traites : La table traite stocke toutes les données relatives à une traite : l'heure de début, l'heure de fin, la vache, le volume de lait et si la traite s'est bien passée. Chaque traite est identifiée avec un numéro unique.

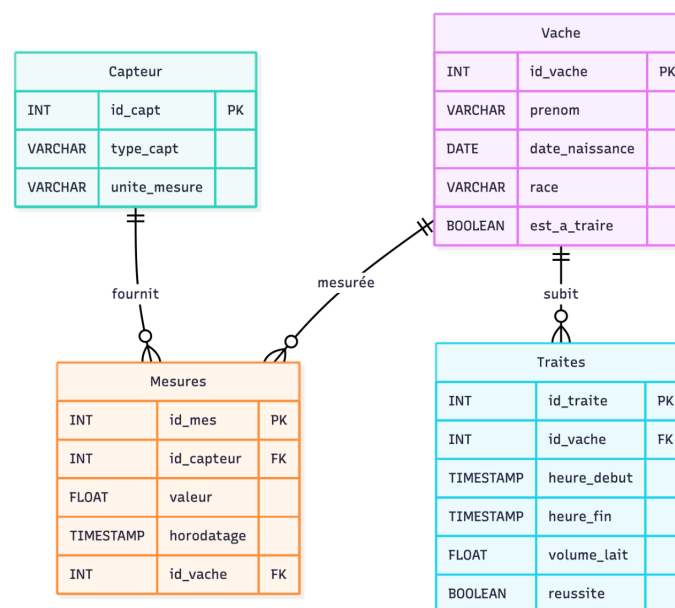


Figure 4 : Schéma de la base de données.

III. Implémentation des microservices

A. Architecture des microservices

L'architecture des microservices est présentée à la Figure 5. Les microservices que nous avons mis en place sont présentés ci-dessous :

Configuration Service : Ce microservice sert à donner la configuration à tous les autres microservice. Utilisant `@EnableConfigServer`, il centralise tous les fichiers de paramètres (.properties). Si le port d'un capteur change, il doit être modifié ici, et tous les services se mettent à jour.

Discovery Service : Permet d'enregistrer tous les microservices sauf celui de configuration. Grâce à @EnableEurekaServer, chaque microservice s'enregistre auprès de lui au démarrage. Cela permet à l'Orchestrateur de trouver les autres services par leur nom sans connaître leur adresse IP exacte. Il permet donc d'avoir une infrastructure plus robuste au changement d'adresse IP de la part des autres microservices.

Data Service : Sert d'interface entre les microservices et la base de données. Il est le seul à communiquer avec elle. Il expose des points d'accès (API REST) pour lire ou modifier l'état des vaches.

Sensor Presence Service : Permet de communiquer avec les capteurs de présence, actuellement il envoie un ID de vache aléatoirement.

Sensor Flow Service : Permet de récupérer les données venant des capteurs de débit. Actuellement il envoie un float tiré aléatoirement entre 0 et 10 qui représente 0 à 10 L/minute. Ce sont des débits réalistes.

Orchestrator Service : C'est le service le plus complexe. Il ne possède pas de base de données propre, mais il pilote les autres.

Il utilise l'outil Feign. Au lieu d'écrire de longues requêtes HTTP, cet outil nous permet d'utiliser des interfaces comme DataServiceClient. Spring génère automatiquement le code pour appeler le data-service via le réseau. Le système surveille périodiquement l'état de la salle de traite pour automatiser le passage d'une étape à l'autre (Entrée -> Traite -> Sortie).

Comme l'Orchestrateur ne possède pas les entités de la base de données (les classes @Entity sont dans le service data), il utilise des Data Transfer Objects (DTO), par exemple VacheDTO ou MesureDTO. Cela évite d'exposer la structure réelle de la base de données et réduit la taille des messages JSON échangés sur le réseau.

L'Orchestrateur maintient un objet ParlorState qui agrège les informations provenant de plusieurs microservices pour qu'elles soient visualisées sur le dashboard.

Lorsqu'un événement (temporel ou capteur) réveille l'orchestrateur, il demande une info au presence-service. Il traite alors l'information. Il traite l'information (avec une comparaison de seuils et la vérification du statut estATraire). Il envoie finalement une commande de mise à jour au data-service avec le résultat.

C'est donc le cœur de l'intelligence du système, c'est lui qui fait toute la logique entre les microservices. Une interface graphique est aussi présente pour vérifier plus facilement que la logique est bien appliquée.

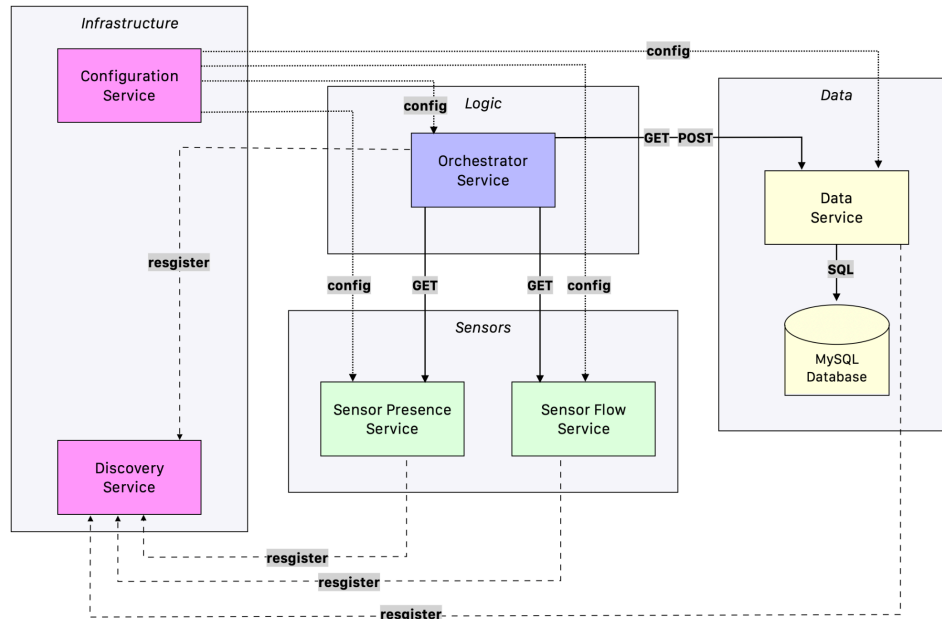


Figure 5 : Schéma des microservices.

B. Interface graphique

L'interface se segmente en trois zones :

- Une représentation de la salle de traite où chaque poste affiche l'état de la machine à états (FREE, MILKING, ENTERING, EXITING), le nom de la vache identifiée et son débit de lait actuel.
- Une liste des vaches en attente (récupérées depuis le Data-Service) permettant de visualiser d'un coup d'œil les animaux restant à traire.
- Un panneau de contrôle permettant entre autres de réinitialiser le système, envoyant des requêtes POST vers l'Orchestrateur pour modifier l'état global du système et de la base de données.



Figure 6 : Salle de traite vide.

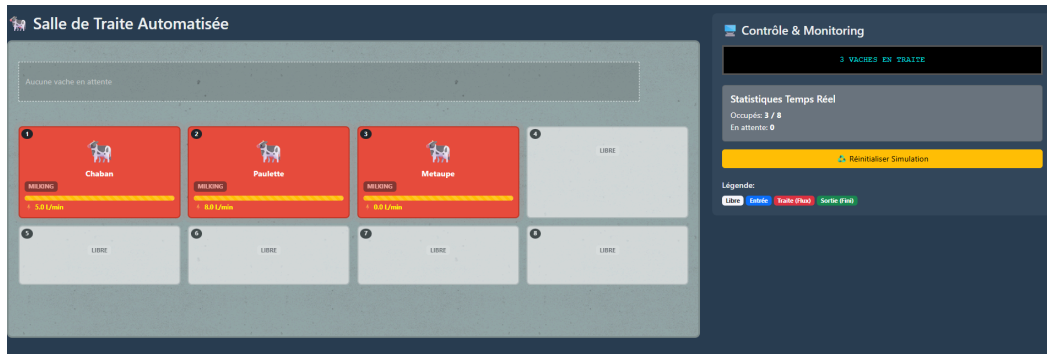


Figure 7 : Salle de traite en cours de traite.



Figure 8 : Salle de traite en fin de traite

C. Exemples de scénarios

Scénario 1 : Arrivée et identification d'une vache

Dans ce scénario, une vache entre dans la salle. Le système doit vérifier si elle est autorisée à être traite. L'orchestrateur appelle Presence-Service (avec la méthode checkPresence()). Le capteur renvoie l'ID de la vache. L'orchestrateur appelle le Data-service pour récupérer les informations de la vache concernée et lit l'attribut est_a_traire. S'il est *true*, l'orchestrateur change l'état de la stalle (le compartiment cloisonné réservé à l'animal) en ENTERING. L'interface affiche alors le nom de la vache avec une icône orange.

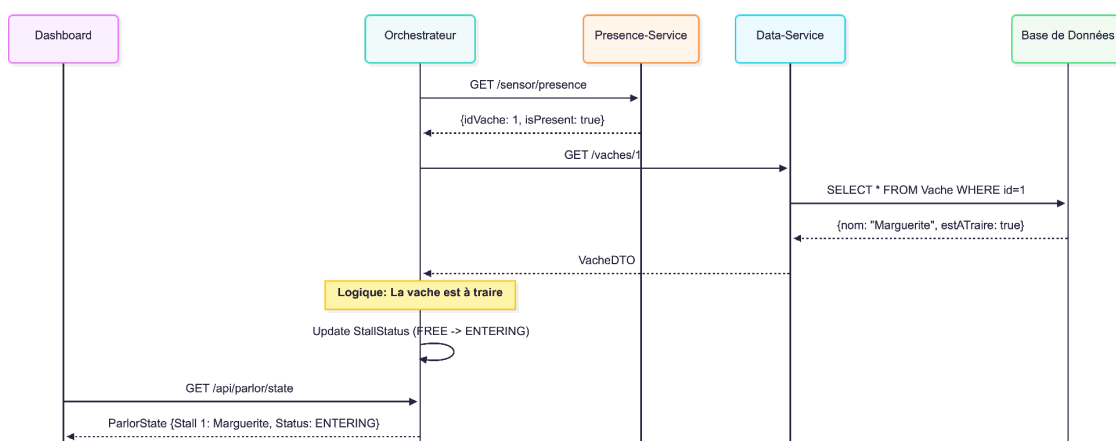


Figure 9 : Diagramme de séquence illustrant le scénario 1.

Scénario 2 : Fin de traite automatique

Dans ce scénario, le système surveille le débit. Quand il devient trop faible, il arrête la traite et enregistre le résultat dans la base de données. L'orchestrateur interroge périodiquement le Flow-Service. Il reçoit une valeur (par exemple, 4 L/min). Le seuil de fin de traite est fixé à 5 L/min. Comme le débit reçu est inférieur au seuil, l'orchestrateur décide d'arrêter la traite. L'orchestrateur envoie un ordre (fictif) de retrait de la griffe. Il appelle Data-Service pour créer une ligne dans la table Traites (volume final, heure de fin, réussite=true) et pour passer est_a_traire à false (avec la méthode setTraiteStatus). La stalle repasse en vert et se libère (FREE).

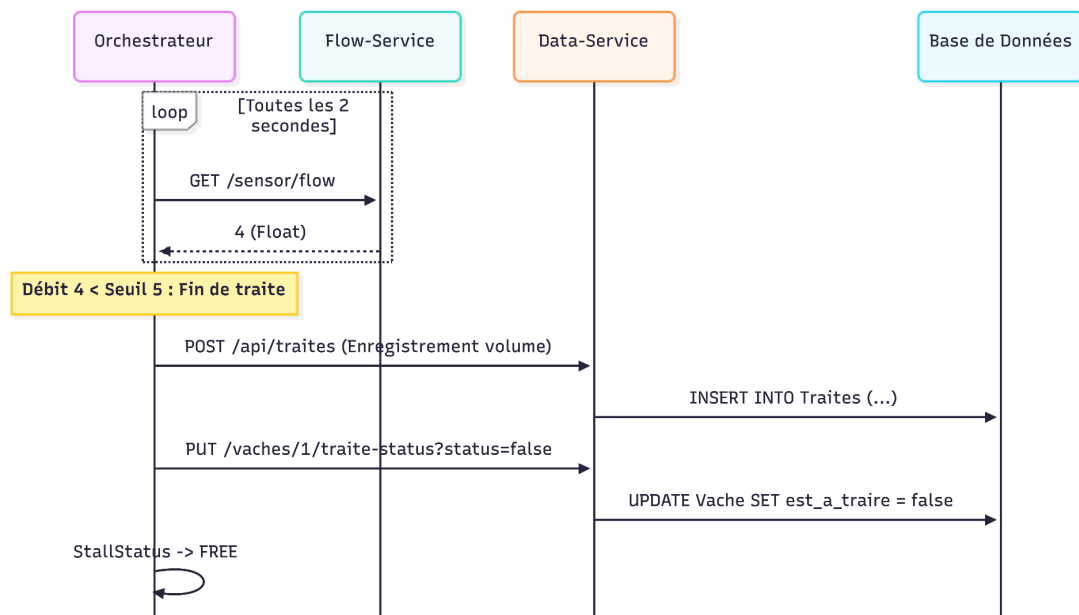


Figure 10 : Diagramme de séquence illustrant le scénario 2.

IV. Mise en place de GitHub et GitHub Action

Nous avons mis en place Github Action workflow, qui pour tous nos microservices, crée une machine virtuelle Ubuntu et exécute mvn package. Nous avons fait un fichier de configuration par microservice qui s'active si le microservice ou le fichier de configuration change.

Tous les fichiers de configuration et notre code sont disponibles sur notre repository GitHub: <https://github.com/jploubejac/INSA-5ISS-Vacow>.

V. Conclusion

Ce projet nous a permis de mettre en œuvre une architecture orientée microservices dédiée à l'automatisation et au suivi d'une salle de traite laitière. En exploitant l'écosystème Spring Boot, le système est modulable et évolutif, ce qui est important si l'on souhaite intégrer de nouveaux capteurs ou actionneurs.

Chaque microservice a été conçu pour isoler une fonction précise, qu'il s'agisse de la gestion du cheptel, du monitoring du débit ou de la détection de présence. Les microservices communiquent avec des API REST. L'utilisation du serveur de découverte Eureka et de Spring Cloud Config permet de gérer l'enregistrement automatique des services et une configuration centralisée.